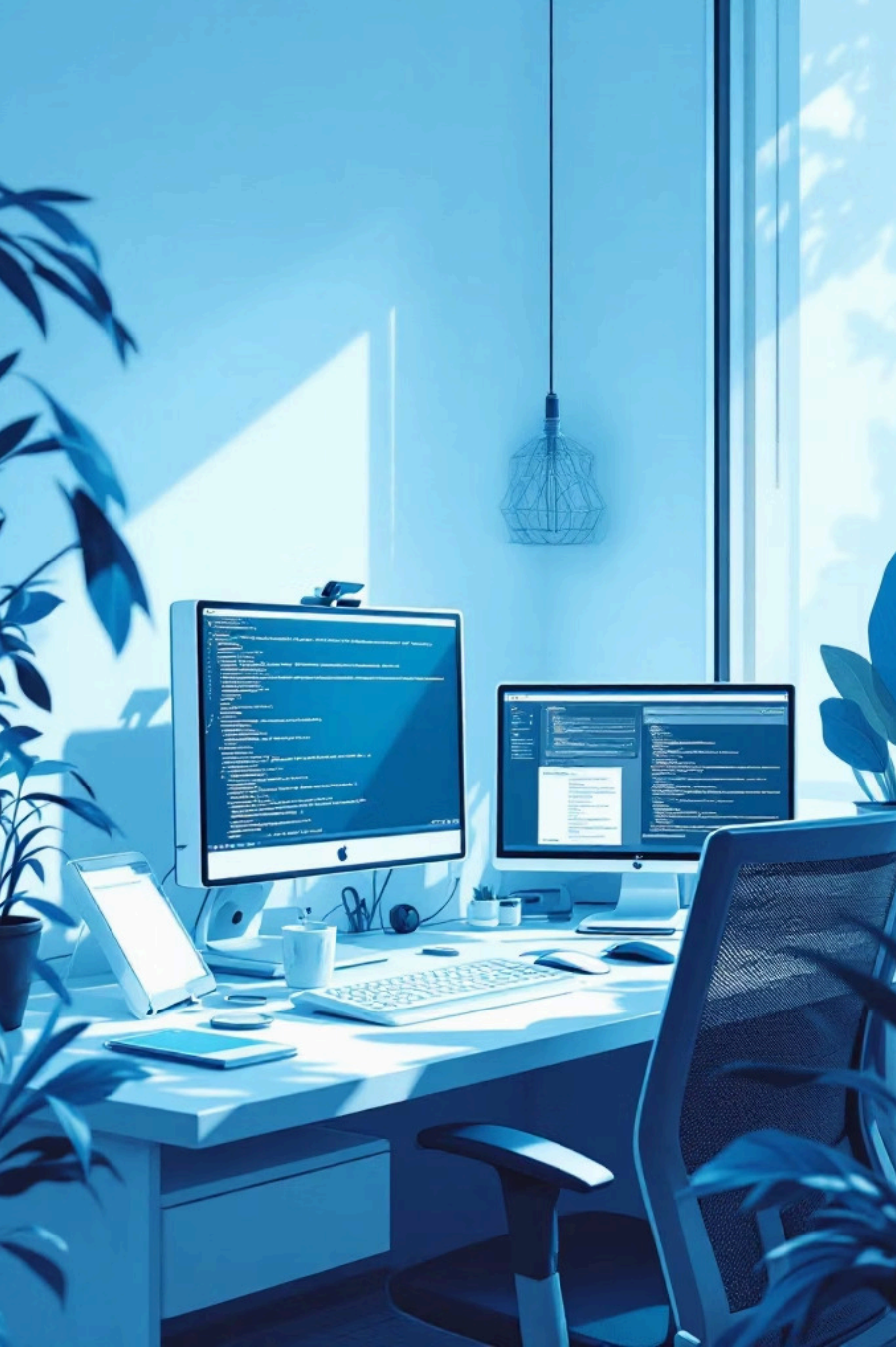




Mastering R: Conditions, Loops, and Operators

Unlock the power of R programming by mastering its fundamental control structures and operators. These essential tools form the backbone of any sophisticated data analysis, enabling you to write efficient, logical, and dynamic code that adapts to your data's characteristics.

The Building Blocks: Operators in R

Arithmetic Operators

Perform mathematical calculations and numerical operations on your data

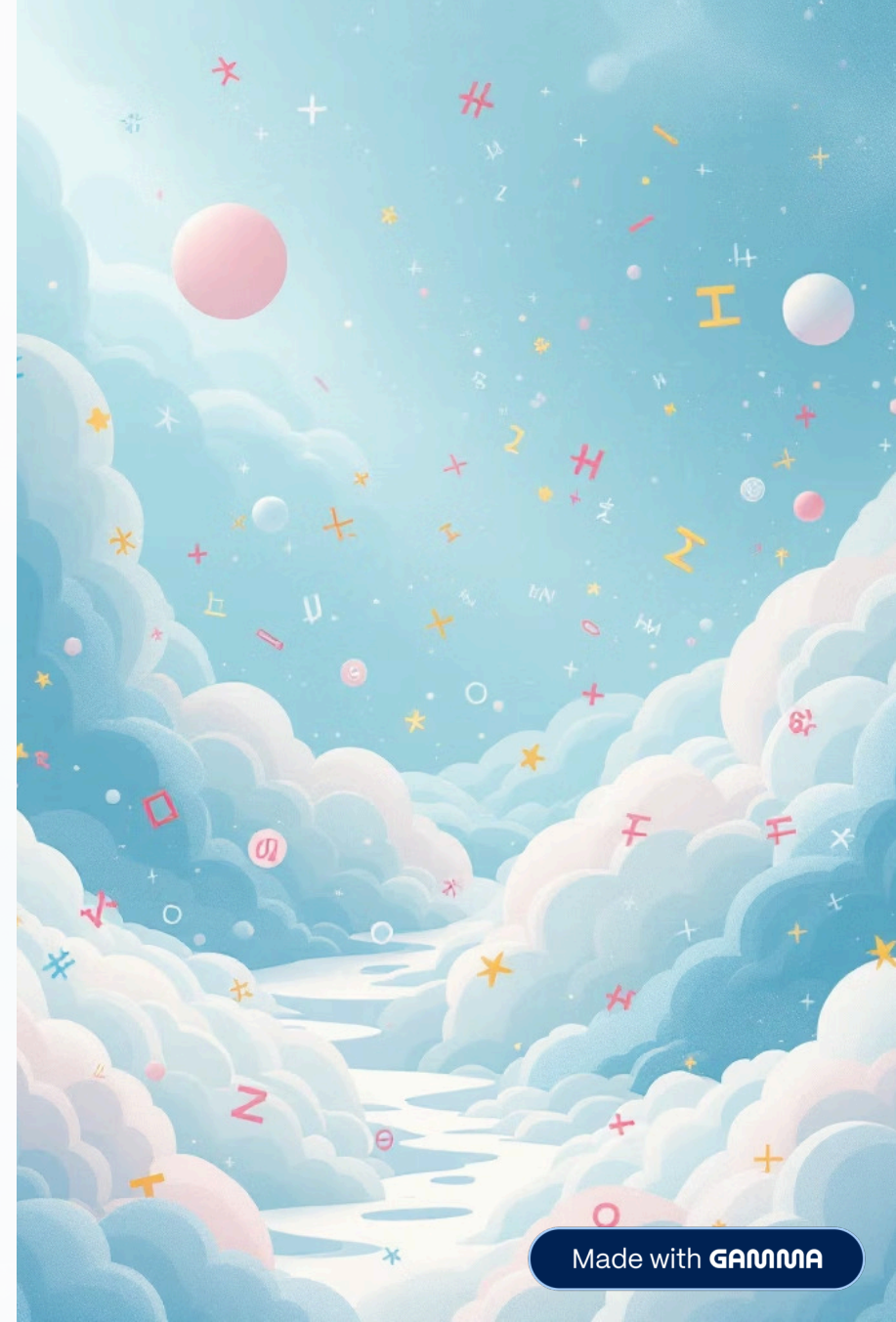
Comparison Operators

Compare values and make logical decisions based on relationships

Logical Operators

Combine multiple conditions to create complex decision logic

Operators are the fundamental tools that allow R to perform calculations, compare values, and make decisions. Understanding these operators is essential for writing effective R code that processes data intelligently and responds appropriately to different conditions.



Arithmetic Operators: The Foundation of Calculation

Arithmetic operators perform mathematical operations on numeric values. These fundamental operations form the basis of all numerical computations in R, from simple calculations to complex statistical analyses.

1	Addition: + 5 + 3 results in 8 Combines two values to produce their sum
2	Subtraction: - 10 - 4 results in 6 Calculates the difference between two values
3	Multiplication: * 6 * 7 results in 42 Multiplies two values to produce their product
4	Division: / 20 / 5 results in 4 Divides the first value by the second
5	Exponentiation: ^ or ** 2 ^ 3 results in 8 Raises the first value to the power of the second
6	Modulo (Remainder): %% 10 %% 3 results in 1 Returns the remainder after division
7	Integer Division: %/% 10 %/% 3 results in 3 Returns the whole number of times division occurs

Comparison Operators: Making Decisions

Comparison operators evaluate relationships between values, returning `TRUE` or `FALSE`. These operators are essential for conditional logic, allowing your code to respond differently based on data characteristics.



Equal to: `==`

`5 == 5` returns **TRUE**

Tests whether two values are identical



Not equal to: `!=`

`5 != 3` returns **TRUE**

Tests whether two values differ



Less than: `<`

`3 < 5` returns **TRUE**

Tests whether the first value is smaller



Less than or equal to: `<=`

`5 <= 5` returns **TRUE**

Tests whether the first value is smaller or equal



Greater than: `>`

`7 > 5` returns **TRUE**

Tests whether the first value is larger



Greater than or equal to: `>=`

`5 >= 5` returns **TRUE**

Tests whether the first value is larger or equal

Logical Operators: Combining Conditions

Logical operators combine multiple boolean conditions, enabling sophisticated decision-making in your code. These operators allow you to create complex logical expressions that evaluate multiple criteria simultaneously.



AND: `&` and `&&`

`TRUE & TRUE` is **TRUE**

`TRUE & FALSE` is **FALSE**

Element-wise (`&`): Applies to each element in vectors

Short-circuit (`&&`): Evaluates only until result is determined



OR: `|` and `||`

`TRUE | FALSE` is **TRUE**

`FALSE | FALSE` is **FALSE**

Element-wise (`|`): Applies to each element in vectors

Short-circuit (`||`): Evaluates only until result is determined



NOT: `!`

`!TRUE` is **FALSE**

`!FALSE` is **TRUE**

Inverts the boolean value

Useful for negating conditions

Practical Examples

AND operator examples

`(5 > 3) & (10 < 15)` # TRUE - both conditions are TRUE

`(5 > 3) & (10 > 15)` # FALSE - second condition is FALSE

OR operator examples

`(5 > 3) | (10 > 15)` # TRUE - first condition is TRUE

`(5 < 3) | (10 > 15)` # FALSE - both conditions are FALSE

NOT operator examples

`!(5 == 3)` # TRUE - negates FALSE

`!(5 == 5)` # FALSE - negates TRUE

Conditional Execution: The if Statement

The `if` statement allows code to execute only when a specified condition evaluates to `TRUE`. This fundamental control structure enables your programs to make decisions and respond differently based on data characteristics.

Basic Syntax Structure

```
if (condition) {  
  # Code to execute if condition is TRUE  
} else {  
  # Code to execute if condition is FALSE  
}
```

01	02	03
Evaluate Condition	Execute <code>if</code> Block	Execute <code>else</code> Block
The condition inside parentheses is tested and must return a logical value	If condition is <code>TRUE</code> , run the code within the first curly braces	If condition is <code>FALSE</code> , run the code within the optional else block

Complete Example with Multiple Conditions

```
# Example: Grade determination  
score <- 85  
  
if (score >= 90) {  
  grade <- "A"  
  print("Excellent work!")  
} else if (score >= 80) {  
  grade <- "B"  
  print("Good job!")  
} else if (score >= 70) {  
  grade <- "C"  
  print("You passed.")  
} else {  
  grade <- "F"  
  print("Needs improvement.")  
}  
  
print(paste("Your grade is:", grade))
```

Output: "Good job!" and "Your grade is: B"

❏ **Key Points:** The `else if` clause allows testing multiple conditions sequentially. The `else` block is optional and catches all remaining cases. Conditions are evaluated in order, and once one is `TRUE`, subsequent conditions are skipped.